SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Hoang-Nam Tran

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Performancevorhersage iteratorbasierter Datenbanksysteme mit Entscheidungsbäumen

Author:	Hoang-Nam Tran
Examiner:	Prof. Dr. Thomas Neumann
Supervisor:	Maximilian Rieger, M.Sc.
Submission Date:	27.04.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 27.04.2025 Hoang-Nam Tran

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Section . . . 1
1.1.1 Subsection . . . 1

2 Theoretical Background **3**
2.1 Databases and Query Execution . . . 3
2.1.1 Iterator-Based Execution (PostgreSQL) . . . 3
2.1.2 Compiled Query Execution (Umbra) . . . 3
2.1.3 Pipelines and Pipeline Breakers . . . 3
2.2 Database Performance Prediction . . . 3
2.2.1 Benchmarking . . . 3
2.2.2 Error Metrics . . . 3
2.2.3 Traditional Cost Models and Learned Approaches . . . 3
2.2.4 Neural Networks, Decision Trees, and T3 . . . 3
2.3 Related Work . . . 3
2.4 ——————— . . . 3
2.5 Database Basics . . . 3
2.6 Performance Prediction . . . 3
2.7 Benchmarking . . . 4
2.8 Cost Models . . . 4
2.9 Decision Trees . . . 4
2.10 PostgreSQL . . . 4
2.11 Umbra . . . 4
2.12 T3 . . . 4
2.13 Section . . . 4
2.13.1 Subsection . . . 4

3 Methodology **6**
3.1 Problem Statement and Prediction Target . . . 6

3.2 Data and Preprocessing 6
3.3 Implementation 7
3.3.1 Components Taken from T3 7
3.3.2 Extensions for PostgreSQL Parsed Plans 7
3.4 Other Performance Prediction Implementations 8
3.5 Training Setup 8
3.6 Section 8
3.6.1 Subsection 8

4 Evaluation **10**
4.1 Ablation Study 10
4.1.1 PostgreSQL-native Features vs. Direct Transfer 10
4.1.2 Actual vs. Estimated Cardinalities 10
4.2 Section 10
4.2.1 Subsection 10

5 Future Work **12**
5.1 Section 12
5.1.1 Subsection 12

6 Conclusion **14**
6.1 Section 14
6.1.1 Subsection 14

Abbreviations **16**

List of Figures **17**

List of Tables **18**

Bibliography **19**

1 Introduction

1.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}`, `\ac{TUM}` ⇒ Technical University of Munich (TUM), TUM

For more details, see the documentation of the `acronym` package[1].

1.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

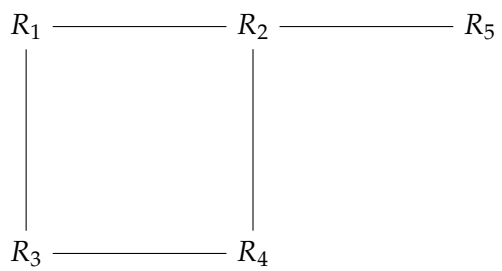

Figure 1.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

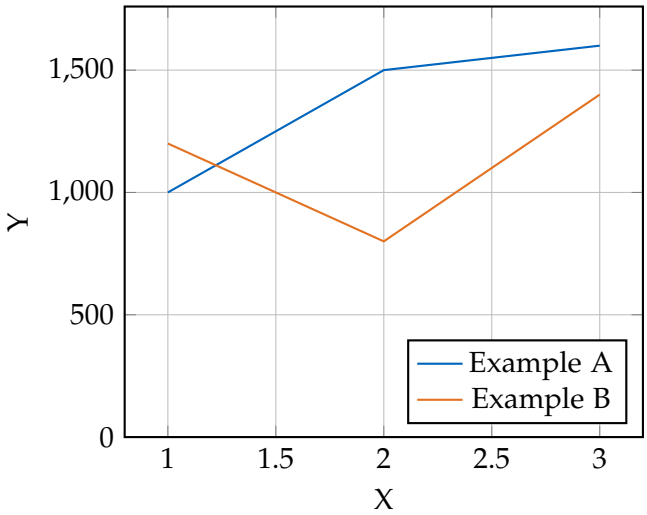

Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Theoretical Background

2.1 Databases and Query Execution

2.1.1 Iterator-Based Execution (PostgreSQL)

2.1.2 Compiled Query Execution (Umbra)

2.1.3 Pipelines and Pipeline Breakers

2.2 Database Performance Prediction

2.2.1 Benchmarking

sowas auch schreiben wie ssb, tpch, etc.

2.2.2 Error Metrics

2.2.3 Traditional Cost Models and Learned Approaches

2.2.4 Neural Networks, Decision Trees, and T3

2.3 Related Work

2.4 ——————————

2.5 Database Basics

operator type, estimated/actual cardinalities, widths, and runtimes

2.6 Performance Prediction

draft: beschreiben, was performance prediction ist, was die ziele sind, etc. benchmarking, etc. Cost Model beschreiben

2.7 Benchmarking

timed queries, etc.

2.8 Cost Models

2.9 Decision Trees

2.10 PostgreSQL

2.11 Umbra

2.12 T3

2.13 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

2.13.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 2.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

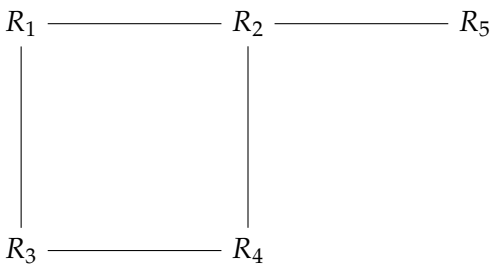

Figure 2.1: An example for a simple drawing.

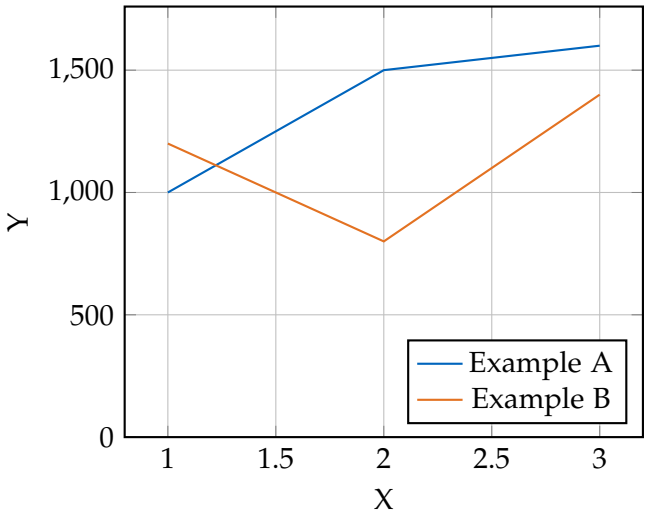

Figure 2.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 2.3: An example for a source code listing.

3 Methodology

3.1 Problem Statement and Prediction Target

In this thesis, we address the transfer of the T3 runtime prediction approach from Umbra to PostgreSQL parsed plans. The central methodological goal is to preserve T3's core modeling principles. Upon that, we adapt plan conversion and feature extraction to PostgreSQL-specific plan information.

Also, the prediction target is total query runtime. Following T3, each query plan is decomposed into pipelines and a per-tuple runtime is predicted for each pipeline. Afterwards, we aggregate the per-tuple and then the pipeline runtimes to obtain the final query-runtime estimate.

3.2 Data and Preprocessing

Regarding the training data, we use the public Zero-Shot benchmark data introduced with the Zero-Shot Cost Model. The benchmark is particularly suitable for this thesis because it consists of real-world datasets (e.g., IMDB) and established benchmark workloads (e.g., SSB, TPC-H and JOB). Overall, the benchmark contains 20 databases with large variation in tables, columns, and foreign-key relationships [HB22].

Having this multitude of databases allows us to test the generalization capabilities of our model. Similar to the T3 paper, we can train the model on all but one databases and test on the remaining one. Moreover, as the widely used JOB workload is included, we can compare our performance predictions to other implementations later on. Further, PostgreSQL is the intended database system of our implementation, which avoids the effort of building a separate PostgreSQL benchmark.

Following the benchmark construction described in the paper, queries are generated per database and executed on a PostgreSQL DBMS (v12) on c8220 nodes on the CloudLab platform. Then, they are stored as raw traces (i. e., the output of `EXPLAIN ANALYZE`). Next, the authors preprocess the queries to remove very long-running queries (queries above 30 s are excluded) to keep the benchmark practical and reproducible [HB22]. A further preprocessing step is to parse the raw traces into structured plan representations. These parsed plans (15000 per database) contain the operator nodes

found in the raw traces, but organize them in a tree structure with node-wise attributes. For every node, e.g., operator type, estimated/actual cardinalities, widths, and runtimes are stored. The tree structure can be used to create pipelines and the node information will be used to train the model. In the upcoming sections, we will describe these key implementation details.

Consequently, the parsed plans offer a good starting point for our pipeline and we can begin without further preprocessing.

3.3 Implementation

3.3.1 Components Taken from T3

T3 is our source model and we transfer its core prediction logic from Umbra to PostgreSQL parsed plans. Hence, we adopt its three central design principles: a pipeline-based query plan representation, tuple-centric prediction targets, and gradient-boosted decision trees for runtime prediction [RN25].

We preserve T3's learning and aggregation mechanics: query runtime is obtained by summing predicted pipeline runtimes, each derived from a per-tuple prediction scaled to the pipeline input cardinality as in the original T3 setup.

Training follows the tuple-centric T3 recipe implemented in our codebase (e.g. `src/zeroshot/training_zeroshot_tpch_holdout.py`): we build per-pipeline training rows, transform per-tuple targets with $-\log(t)$, and fit a LightGBM gradient-boosted tree model under a fixed protocol (objective, number of boosting rounds, train/validation split by query).

PostgreSQL parsed plans are first converted into a T3-compatible internal plan and pipeline layout in `src/zeroshot/zeroshot_to_t3.py` (operator mapping, pipeline breakers, node identifiers, and plan-level metadata). This step supplies the pipeline decomposition and operator graph that the rest of the T3-style pipeline still assumes; feature values themselves are filled by our PostgreSQL-specific extension described below.

For ablation only, we additionally evaluate a *direct transfer* variant that keeps the original Umbra-oriented `FeatureMapper` (`src/features.py`) on the converted plans; we report this comparison in Subsection 4.1.1.

3.3.2 Extensions for PostgreSQL Parsed Plans

Our main system replaces Umbra-oriented pipeline features with a dedicated PostgreSQL encoding. During conversion, each plan node carries a `pg` copy of the Zero-Shot `plan_parameters` payload. The `PgFeatureMapper` in `src/pg_features.py` then

builds one fixed-length feature vector per pipeline from those fields: aggregated cardinalities, planner cost and startup estimates, tuple width, planned worker counts, operator-type counts, simple statistics over scan filters, and an additional operator-level block that sums PG-native signals over operators in pipeline order. The mapper exposes the same interface as T3's feature mapper (`get_pipeline_estimation_matrix`, `get_pipeline_scan_sizes`) so the existing per-tuple runtime model and training loop stay unchanged.

Observed operator timings from the trace are not used as model inputs; they remain available only for defining training labels and pipeline durations where needed.

3.4 Other Performance Prediction Implementations

Add other implementations that you want to compare to.

3.5 Training Setup

Trees nennen Add auch hier deine system specs, seed, split etc...

3.6 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

3.6.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

[1] `https://ctan.org/pkg/acronym`

Table 3.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

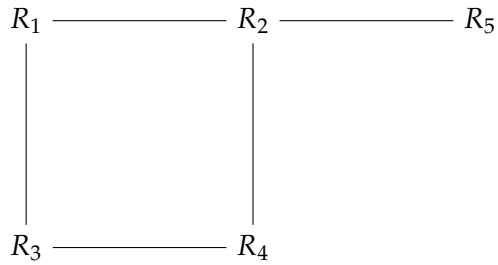

Figure 3.1: An example for a simple drawing.

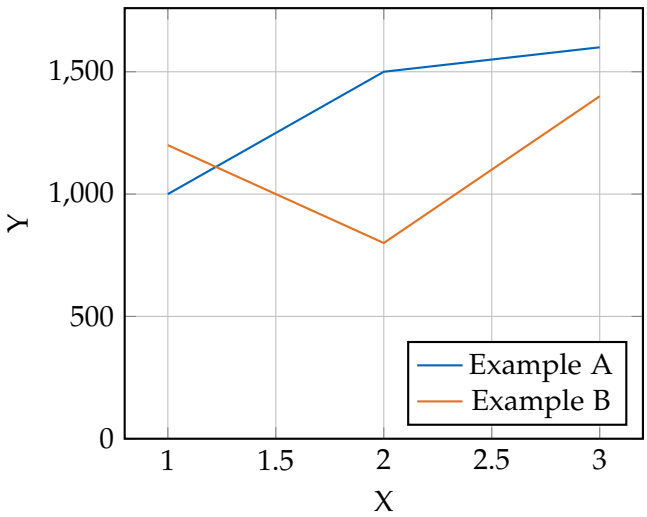

Figure 3.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 3.3: An example for a source code listing.

4 Evaluation

4.1 Ablation Study

4.1.1 PostgreSQL-native Features vs. Direct Transfer

4.1.2 Actual vs. Estimated Cardinalities

4.2 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

4.2.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 4.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

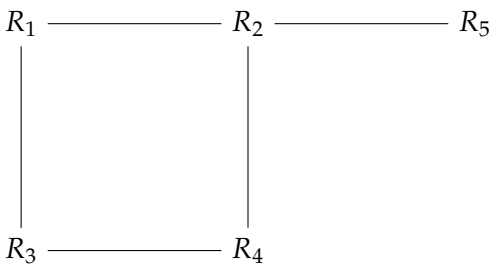

Figure 4.1: An example for a simple drawing.

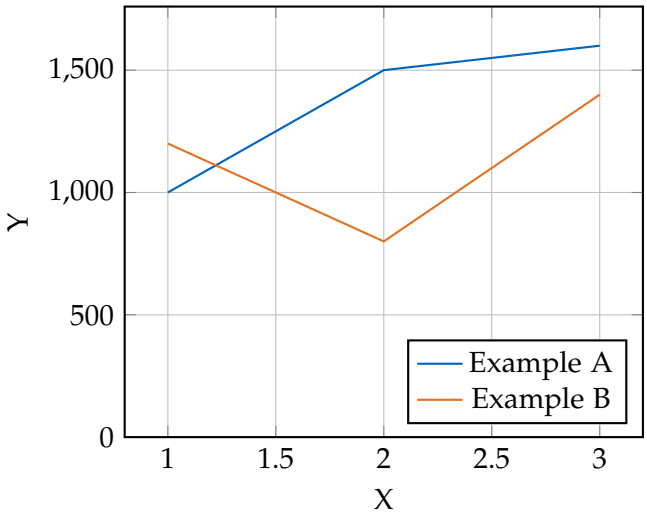

Figure 4.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 4.3: An example for a source code listing.

5 Future Work

Data from Zeroshot kann limitierender faktor sein.

5.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

5.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 5.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

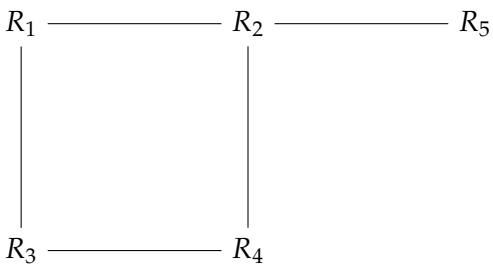

Figure 5.1: An example for a simple drawing.

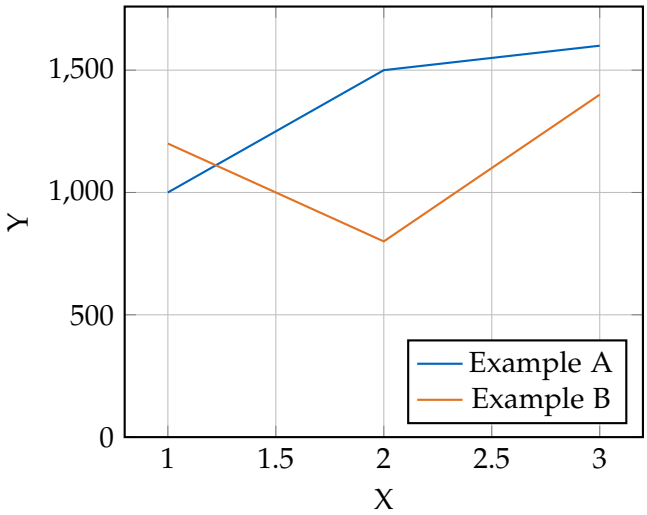

Figure 5.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 5.3: An example for a source code listing.

6 Conclusion

6.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

6.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 6.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

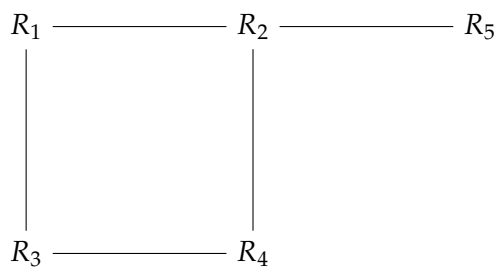

Figure 6.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

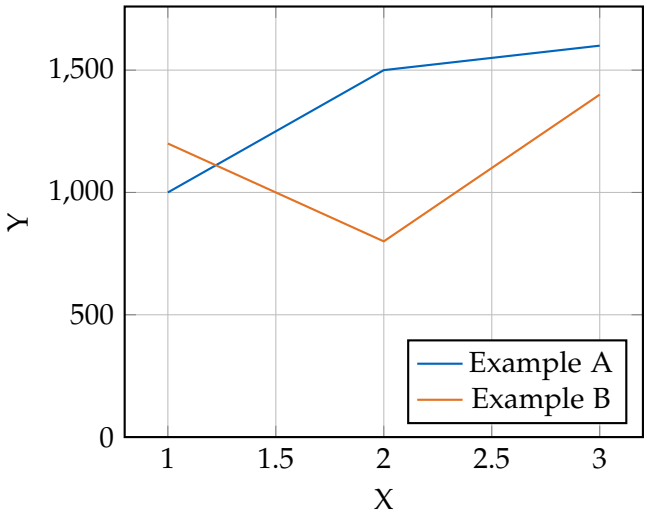

Figure 6.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 6.3: An example for a source code listing.

Abbreviations

TUM Technical University of Munich

List of Figures

1.1 Example drawing 1
1.2 Example plot 2
1.3 Example listing 2

2.1 Example drawing 5
2.2 Example plot 5
2.3 Example listing 5

3.1 Example drawing 9
3.2 Example plot 9
3.3 Example listing 9

4.1 Example drawing 11
4.2 Example plot 11
4.3 Example listing 11

5.1 Example drawing 13
5.2 Example plot 13
5.3 Example listing 13

6.1 Example drawing 14
6.2 Example plot 15
6.3 Example listing 15

List of Tables

1.1 Example table 1

2.1 Example table 4

3.1 Example table 9

4.1 Example table 10

5.1 Example table 12

6.1 Example table 14

Bibliography

[HB22] B. Hilprecht and C. Binnig. "Zero-shot cost models for out-of-the-box learned cost prediction." In: *Proc. VLDB Endow.* 15.11 (July 2022), pp. 2361–2374. ISSN: 2150-8097. DOI: 10.14778/3551793.3551799.

[Lam94] L. Lamport. *LaTeX : A Documentation Preparation System User's Guide and Reference Manual*. Addison-Wesley Professional, 1994.

[RN25] M. Rieger and T. Neumann. "T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees." In: *Proc. ACM Manag. Data* 3.3 (June 2025). DOI: 10.1145/3725364.